

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial y} \left[\frac{\partial y}{\partial z_1} \times \frac{\partial z_1}{\partial w_{11}} \times \frac{\partial w_{11}}{\partial w_{11}} + \frac{\partial y}{\partial z_2} \times \frac{\partial z_2}{\partial w_{11}} \times \frac{\partial w_{11}}{\partial w_{11}} \right]$$

↳ very complex with only 2 layers

Backpropagation → chain diff rule + Memorization

↳ Space the derivation while calculated one time.

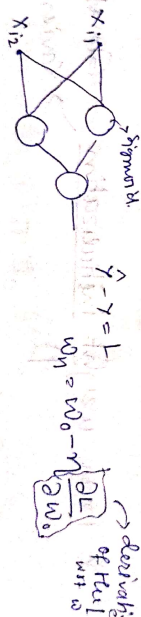
Vanishing Gradient Problem

In ML, the vanishing gradient problem is encountered when training ANNs with GD and backpropagation. In such methods, during each iteration of train each of the NN's weights received an update proportional to the partial derivative of the error function w.r.t. the current weight. In worst case, this may stop the training of NN.

1) $0.1 \times 0.1 \times 0.1 \times 0.1 \times 0.1 = [0.00001] \rightarrow \underline{VGP}$

2) Deep NN → $\square \square \square \square \square$

3) Sigmoid/tanh → Activation fn →



$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial z} \times \frac{\partial z}{\partial w_{11}} \times \frac{\partial w_{11}}{\partial w_{11}}$$

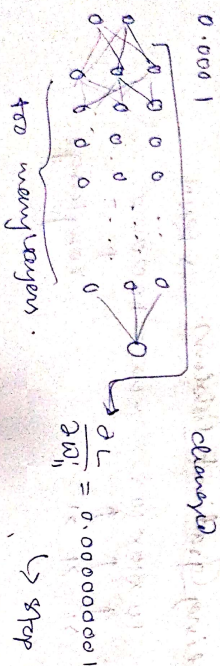
↑ ↑ ↑

0.00001 0.00001 0.00001

did not change

$w_{11} = 1 - 0.01 \times 0.00001$

↳ $w_{11} = 0.99999$



↳ stop training.

How to recognize VGP?

1) Loss focus → epoch → no changes → VGP

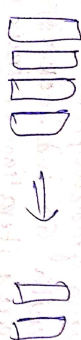
2) weights → graph

weights
↓
loss
↓
error
↓
epoch

epoch
↓
loss
↓
error
↓
epoch

How to handle Vanishing Gradient Problem?

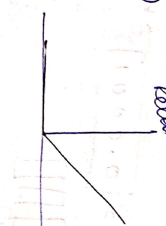
1) Reduce model complexity



→ Problem can't find complex patterns

2) Using ReLU Activation functions

$\max(0, z)$



by using relu → derivative $w_1 = w_0$

3) Proper weight initialization → Xavier

4) Batch normalization → layer

5) Residual Networks → CNN, ResNet

Expanding Gradient Problem → RNN

$w_1 \times 10 \dots 10 \times 10 \approx 100000 \dots$

↑

> 1 > 1 > 1

very large.